Ranveer Thakur Chand

Patricia McManus

ITAI 1378

24 June 2026

Reflective Journal

In this workshop, the CNN uses three convolutional blocks each with a Conv2d, ReLU, and MaxPool2d that grow from 32 to 64, to 128 channels, then flatten into two fully connected layers with 0.5 dropout before the output. The big difference from the MLP in the last workshop is that the MLP flattened the entire image into one long line or vector right away, losing spatial structure. The CNN instead uses 3×3 filters that scan small local patches and share weights across the whole image, so from the start layers naturally pick up edges and textures while deeper layers build those into shapes and objects all without having to do it manually.

The CNN opened at 96.67% validation accuracy on epoch 1, which already beat the 93.33% the MLP, even though the MLP did hit 96.67% but it never started, or landed on it, and the MLP took multiple training sessions, changes, and fixing errors before it was able to reach what the CNN model started with. First, I started by bumping the image size to 256×256, then lowering the learning rate to 0.0005, and capping the epochs at 7, it hit 100% by the end. The reason I did these changes was because increasing the image size gave the model more area to work with, and the learning rate I didn't want to jump straight to 0.0001, so I started with 0.0005. As for the epochs I noticed it hit 0.9667 validation accuracy early in the epochs but it started fluctuating in the last couple epochs, this was most likely due to overfitting or memorization. The few early misclassifications I had were chihuahuas and muffins that look similar at a low level, same round shape, warm color, bumpy texture. But the CNN was able to distinguish those easily.

The CNN outperformed the MLP from epoch 1 and needed minimal manual tweaking to get a 100% validation accuracy. The MLP took several rounds of changing optimizers, learning rates, and epoch counts just to hit 93.33%, and it wasn't that stable. I did notice individual CNN epochs do take slightly longer, but it converges faster and the built-in data augmentation random flips and rotations gives the model more variety to learn from without needing more images, so overall the CNN model completely outperformed the MLP.

The main issue was a UserWarning about num_workers being too high, the DataLoader had it set to 4, which was more than my system or more accurately Google Colab specs could handle. I dropped it to 2 on both loaders fixing it. The other call was stopping at epoch 7 once the model hit 100% validation accuracy, since pushing further made the model overfit. The 0.5 dropout in the classifier helped with that too by keeping the network from relying too heavily on any one path during training, Gemini helped me understand this part.

This exact CNN architecture used in this workshop applies directly to medical imaging, detecting tumors, classifying skin lesions, and flagging retinal issues. As well as retail visual search and agricultural industry for crop disease. Transfer learning, covered in this workshop, makes this workshop more realistic in practice because that's the standard practice in the real

world. Instead of training from scratch with millions of images, you fine-tune a pretrained ResNet or EfficientNet on a smaller dataset and still get great results.

The main concerns are bias, privacy, and explainability. If the training data isn't representative, the model fails on underrepresented cases. In something like a medical diagnosis, that's a really big problem, not just an oopsie. Once deployed, a vision model is also collecting data on real people and environments, often without clear consent, and could be misused for uses it wasn't designed for. CNNs also don't explain their decisions, which is a problem where someone could directly be affected by the output. The fix that I can think of is diverse training data, honest and proper documentation of limitations, and thinking of possible misuse of the model pre deployment or even during development.